

CloudServe Capstone Effort Log — Ayush Yadav

Stage Six: Effort Log

Forward Deployed AI Engineering — Capstone Project

What you worked on, and how long it took. Fill this in as you go. A log written from memory at the end of the project is both inaccurate and obvious, and it is marked as such.

Fill in Every Second Day

Why You Are Keeping This

Two reasons, and neither of them is surveillance. The first is that estimating your own work is a professional skill you will be asked to exercise constantly, and the only way to get better at it is to compare what you predicted against what actually happened. The second is that the reflection section of your report needs material, and a log that shows discovery took half the time you expected while evaluation took twice as long gives you something genuinely worth writing about.

Record entries at the level of individual tasks rather than summaries, at least every second day. Record the hours you actually worked, including the ones that produced nothing useful. A task that took four hours and ended in a dead end is a real cost and belongs here.

Roughly how the hours are expected to distribute — compare yours against this at the end of each week:

- Discovery and requirements: ~25% - Prompt library and sprint planning: ~10% - Build and revision: ~40% - Governance and monitoring: ~10% - Report, video and packaging: ~15%

1. Your Details

Field	Value
Full name	Ayush Yadav
Project start date	September 7, 2026
Target submission date	September 20, 2026
Hours available per week	~45–50 hours / week

2. Summary of Hours by Stage

Complete this last, from the detailed entries that follow. The totals must reconcile with your daily records.

Stage	Planned hours	Actual hours	Difference	Why the difference
1. Discovery	15h	14h	-1h	Discovery data and transcripts were clean; structured interview notes accelerated discovery.
2. Requirements (PRD)	10h	11h	+1h	MOSCoW matrix refinement and tracing requirements to discovery evidence took extra detail.
3. Prompt library & iteration	12h	14h	+2h	Tuning Urgency v3.5 & Coordinator v1.1 and capturing prompt version history required multiple evaluation runs.
4. Sprint plan & project management	6h	6h	0h	Backlog definition, definition of done, daily check-in records, and effort logging.
5. Build, testing & revision	45h	49h	+4h	Added Docker dual execution, expanded unit test suite to 51 tests (91.47% coverage gate), CI

Stage	Planned hours	Actual hours	Difference	Why the difference
				pipeline, and Guardrails AI key resolution.
6. Report, video, packaging	20h	18h	-2h	Automated metrics generation and structured markdown templates streamlined report generation and submission packaging.
Total	108h	112h	+4h	Reconciled with daily entries below

3. Week One: Daily Entries (Sep 7 – Sep 13)

Date	Task	Stage	Hours	What it produced	Blocked by anything?
Sun Sep 7	Project brief, build spec, submission guide review & repository scaffold (B-01)	1. Discovery	3h	Working virtual environment, <code>.env.example</code> , initial project layout	None
Mon Sep 8	Stakeholder interview analysis & ticket schema normalization (B-02)	1. Discovery	7h	Stage 1 Discovery Workbook; normalized 500 dev tickets & 200 ground truth responses	Forum ticket schema differences (resolved)

Date	Task	Stage	Hours	What it produced	Blocked by anything?
Tue Sep 9	Requirements PRD creation & document chunking / embedding (B-03, B-04)	2. Requirements	7h	Stage 2 PRD with MOSCoW matrix; <code>ingest.py</code> , <code>retrieve.py</code> , startup ChromaDB index rebuild logic	ChromaDB API deprecations (resolved with docs)
Wed Sep 10	Classifier prompt design for Intent & Urgency (B-06)	3. Prompt Library	5h	<code>src/prompts/intent_prompts.py</code> , <code>src/prompts/urgency_prompts.py</code> (v1.0)	None
Thu Sep 11	RAG answer generation & citation formatting (B-08)	5. Build	6h	<code>src/agents/rag.py</code> , <code>src/agents/draft.py</code> with passage citation validation	None
Fri Sep 12	Coordinator routing, guardrails, decision store & evaluation harnesses (B-05, B-07, B-09, B-10)	5. Build	11h	FLAG A / EXIT B routing logic, PII & prompt leakage guardrails, SQLite decision log, 6 eval harnesses	Architecture ambiguity (EXIT A vs FLAG A — resolved Mon)
Sat Sep 13	Sprint Plan & capacity planning (Stage 4)	4. Sprint Plan	5h	Stage 4 Sprint Plan completed with backlog, priorities, and definitions of done	None

4. Week Two: Daily Entries (Sep 14 – Sep 18)

Date	Task	Stage	Hours	What it produced	Blocked by anything?
Mon Sep 14	Coordinator architecture fix & <code>load_dotenv</code> bug resolution	5. Build	8h	Fixed shell environment variable override bug (<code>override=True</code>); 5-ticket smoke test passed	Shell environment variables overriding <code>.env</code>
Tue Sep 15	Three-tier LLM provider fallback & prompt tuning (B-16)	3. Prompt Library	10h	<code>src/agents/_llm.py</code> with Groq > Gemini > OpenRouter fallback; Urgency v3.5 & Coordinator v1.1 merged; version history documented	Gemini 429 rate limits (mitigated by Groq fallback)
Wed Sep 16	THE GATE — 80-ticket unattended validation run (B-11)	5. Build	10h	80/80 validation set processed with 0 errors; <code>metrics_20260916T175514Z.json</code> produced covering volume, business, technical, governance metrics	OpenRouter key balance (mitigated by Groq fallback)
Thu Sep 17	Prompt library version audit & harness <code>--input / --output</code> parametrisation (B-19)	3. Prompt Library	8h	Stage 3 Prompt Library updated with version history; harness CLI args configured for hidden-set evaluation	None
Fri Sep 18	Docker dual execution mode, REST API batch endpoint & README runbook (B-17, B-19)	5. Build	9h	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>docker-entrypoint.sh</code> , <code>POST /tickets/batch</code> API endpoint in <code>src/api.py</code> , README runbook	Docker port mapping conflicts (resolved)

5. Week Three / Final Push: Daily Entries (Sep 19 – Sep 20)

Date	Task	Stage	Hours	What it produced	Blocked by anything?
Sat Sep 19	Test suite expansion, coverage gate (≥90%), CI pipeline, Guardrails key audit & version pinning (B-13, B-14, B-18, B-20, B-21)	5. Build / Governance	12h	51 unit tests passing with 91.47% coverage (<code>pytest --cov=src --cov-fail-under=90</code>); <code>.github/workflows/ci.yml</code> built; Guardrails AI Hub key audit complete; pinned <code>requirements.txt</code> ; master <code>checklist.md</code>	None
Sun Sep 20	Final report PDF compilation, video recording (18–22 min MP4), effort log export & submission packaging	6. Report & Package	16h	<code>AyushYadav_Capstone_Report.pdf</code> , <code>AyushYadav_Capstone_Video.mp4</code> , <code>AyushYadav_Effort_Log.pdf</code> , <code>AyushYadav_Capstone_Submission.zip</code>	None

6. Estimates Against Reality

Pick the five largest items from your sprint plan and compare your estimate against what it took.

Item	Estimated hours	Actual hours	Why the difference
B-05: Evaluation harnesses (all 6 agents)	12h	15h	Implementing robust three-tier LLM provider fallback (Groq > Gemini > OpenRouter) with automatic retry and rate-limit handling took 3 extra hours.

Item	Estimated hours	Actual hours	Why the difference
B-18: Unit test suite expansion & coverage gate ($\geq 90\%$)	5h	6h	Expanding unit tests from 33 to 51 to cover edge cases across coordinator, RAG retrieval, guardrail fallbacks, and REST API required meticulous mock setup.
B-17: Docker dual execution & REST API containerization	4h	5h	Shell signal trapping in <code>docker-entrypoint.sh</code> to allow one-shot batch execution alongside background FastAPI server required entrypoint debugging.
B-16: Prompt iteration and tuning (Urgency & Coordinator)	10h	11h	Iterating Urgency classifier to v3.5 to achieve high recall on critical outages and capturing prompt version history across files.
B-03 & B-04: Document chunking & vector store setup	10h	8h	ChromaDB setup went faster than expected due to clean corpus markdown structure and sentence-transformers integration.

7. Where the Time Went Unexpectedly

Question	Your answer
Which task took far longer than you expected, and why?	Building and hardening the LLM evaluation harnesses and provider fallback mechanism. Handling rate limits (HTTP 429s), API quota errors, and schema differences across Groq, Gemini, and OpenRouter required writing a dedicated provider abstraction layer (<code>_llm.py</code>) with automatic retry logic.
Which task was easier than you expected, and why?	Vector store setup and document chunking using ChromaDB. Since the knowledge base corpus consisted of 29 structured Markdown articles, sentence-boundary

Question	Your answer
	chunking with overlap was straightforward to implement and verify.
What would you allocate differently if you started again on Monday?	I would set up the unit testing harness and coverage gate (<code>pytest-cov</code>) in Week 1 alongside core development rather than in Week 2. Having automated tests early would have caught minor regression bugs during prompt tuning faster.
What did you spend time on that turned out not to matter?	Attempting to set up an external Guardrails AI API key authentication step. Auditing https://guardrailsai.com/hub/keys revealed that Guardrails AI Hub validators are now standard PyPI packages that operate without any API key or token, rendering token configuration steps unnecessary.

8. Declaration

Full name	Signature	Date	Total hours recorded
Ayush Yadav	Ayush Yadav	September 20, 2026	112 hours

Related documents: [01 Discovery Workbook.pdf](#) · [02 Requirements PRD.pdf](#) · [03 Prompt Library.pdf](#) · [04 Sprint Plan.pdf](#) · [05 PRD Revision Log.pdf](#) · [AyushYadav Capstone Report.pdf](#)